

IMPLEMENTATION ROADMAP

DANALIT

An Autonomous AI Trading System for MetaTrader 5

Architecture · Free data sources · Machine learning and RL · Risk management
Backtesting and forward testing · Deployment and monitoring · Continuous learning

With 20 sequential build prompts for Claude Code

Instruments: EURUSD · XAUUSD (Gold) · US100 (NASDAQ 100)

Zero-budget, open-source stack — demo first, cent account second, scale later

Prepared for the Danalit project · July 2026

Trading leveraged products involves substantial risk of loss. This document is an engineering plan, not financial advice.

Contents

1. Executive Summary and Reality Check	4
1.1 What you will have at the end	4
1.2 Total cost of ownership	5
2. Guiding Principles	5
3. System Architecture	5
3.1 Layer responsibilities	6
3.2 The two loops	7
4. Technology Stack	7
5. Hosting Strategy	8
6. Machine Learning Approach	8
6.1 The architecture of the intelligence	8
6.2 Validation doctrine (the part that keeps you honest)	9
7. Data: Sources and Acquisition	9
7.1 Historical price data (free)	9
7.2 News and economic calendar data	10
8. MetaTrader Connectivity	10
9. Risk and Money Management for a \$20 Account	11
9.1 The arithmetic problem with \$20	11
9.2 The solution: a cent account	11
9.3 The risk framework (enforced in code, Prompt 10)	11
9.4 Scaling, profit retention and withdrawals	12
10. Testing Strategy	12
10.1 The four gates	12
10.2 Go-live checklist (all must be true)	12
11. Deployment and Operations	12
12. Continuous Improvement	13
13. The 20 Claude Code Prompts	14

Prompt 1 — Project scaffold, configuration and database	14
Prompt 2 — Historical price data pipeline	14
Prompt 3 — News and economic-calendar collector	15
Prompt 4 — Technical feature engineering	16
Prompt 5 — News sentiment and fundamental features	16
Prompt 6 — Labeling and dataset builder	17
Prompt 7 — Baseline signal models	17
Prompt 8 — Event-driven backtesting engine	18
Prompt 9 — Walk-forward harness and honest optimisation	18
Prompt 10 — Risk manager	19
Prompt 11 — Trade management	19
Prompt 12 — Unified signal / decision engine	20
Prompt 13 — Optional RL exit agent	20
Prompt 14 — MT5 broker gateway	21
Prompt 15 — Live orchestrator	21
Prompt 16 — Trade journal and forward-test analytics	22
Prompt 17 — Monitoring: Telegram bot and dashboard	23
Prompt 18 — Continuous learning: retraining pipeline	23
Prompt 19 — Capital policy: scaling, set-aside and withdrawals	24
Prompt 20 — Hardening, deployment and the go-live runbook	24
 14. Timeline and Milestones	 26
 15. Final Words: Success Criteria and Risk Disclosure	 26

1. Executive Summary and Reality Check

This document is a complete, sequential implementation roadmap for **Danalit**: an autonomous AI trading system that analyses historical and real-time price data together with financial news and macroeconomic events, generates trading signals for EURUSD, XAUUSD (gold) and US100 (NASDAQ 100 CFD), and executes and manages trades on MetaTrader 5 — built entirely with free, open-source tools. The centrepiece is a set of **20 carefully sequenced prompts** (Chapter 13) that you give to Claude Code one at a time. Each prompt produces a working, tested module; together they assemble the full system.

Before the architecture, an honest calibration of expectations. You asked for a realistic plan, and realism is the single biggest determinant of whether this project succeeds.

READ THIS FIRST — THE HARD TRUTHS

1) The overwhelming majority of retail algorithmic traders lose money. Markets are close to efficient; any edge is small, unstable, and must survive spread, slippage and swap costs. 2) No AI — including the models in this plan — can reliably predict prices. What a good system does is find a small statistical tilt, size positions so losing streaks are survivable, and execute with perfect discipline. The discipline is where software genuinely beats humans. 3) A \$20 account will not generate meaningful income for a long time. At an excellent and sustainable 5% per month, \$20 becomes roughly \$36 after one year. The realistic goal of the \$20 phase is to prove the system end-to-end with real money and real execution costs — it is tuition, not income. 4) Expect the first model you train to fail forward testing. That is normal; the walk-forward and journaling loops in this plan exist precisely to iterate past it.

What makes this plan worth executing anyway: the infrastructure you will build — clean data pipelines, leakage-free features, an honest backtester, hard risk limits, full trade journaling, and a monitored execution loop — is exactly the infrastructure that separates the small minority of profitable systematic traders from everyone else. It also scales: the same system that trades \$20 on a cent account trades \$20,000 later with only configuration changes.

1.1 What you will have at the end

- A Python trading system (**Danalit**) that ingests free historical tick/minute data and a self-built news archive, engineers technical and fundamental features, and trains gradient-boosted models with walk-forward validation.
- A FinBERT-based news sentiment engine and an economic-calendar awareness layer (news blackout windows, surprise scoring of releases).
- An event-driven backtester with realistic spread, commission, slippage and swap modelling, plus walk-forward optimisation with overfitting guards.
- A risk manager engineered for very small accounts: fixed-fractional sizing, equity tiers, daily loss limits, and drawdown circuit breakers.
- A live execution loop connected to MetaTrader 5 via the official Python API, with reconciliation, a kill switch, Telegram alerts and a Streamlit dashboard.
- A continuous-learning loop: every decision journaled, scheduled retraining, and champion/challenger model promotion.
- A capital policy module that scales risk with equity and produces a monthly profit set-aside/withdrawal recommendation.

1.2 Total cost of ownership

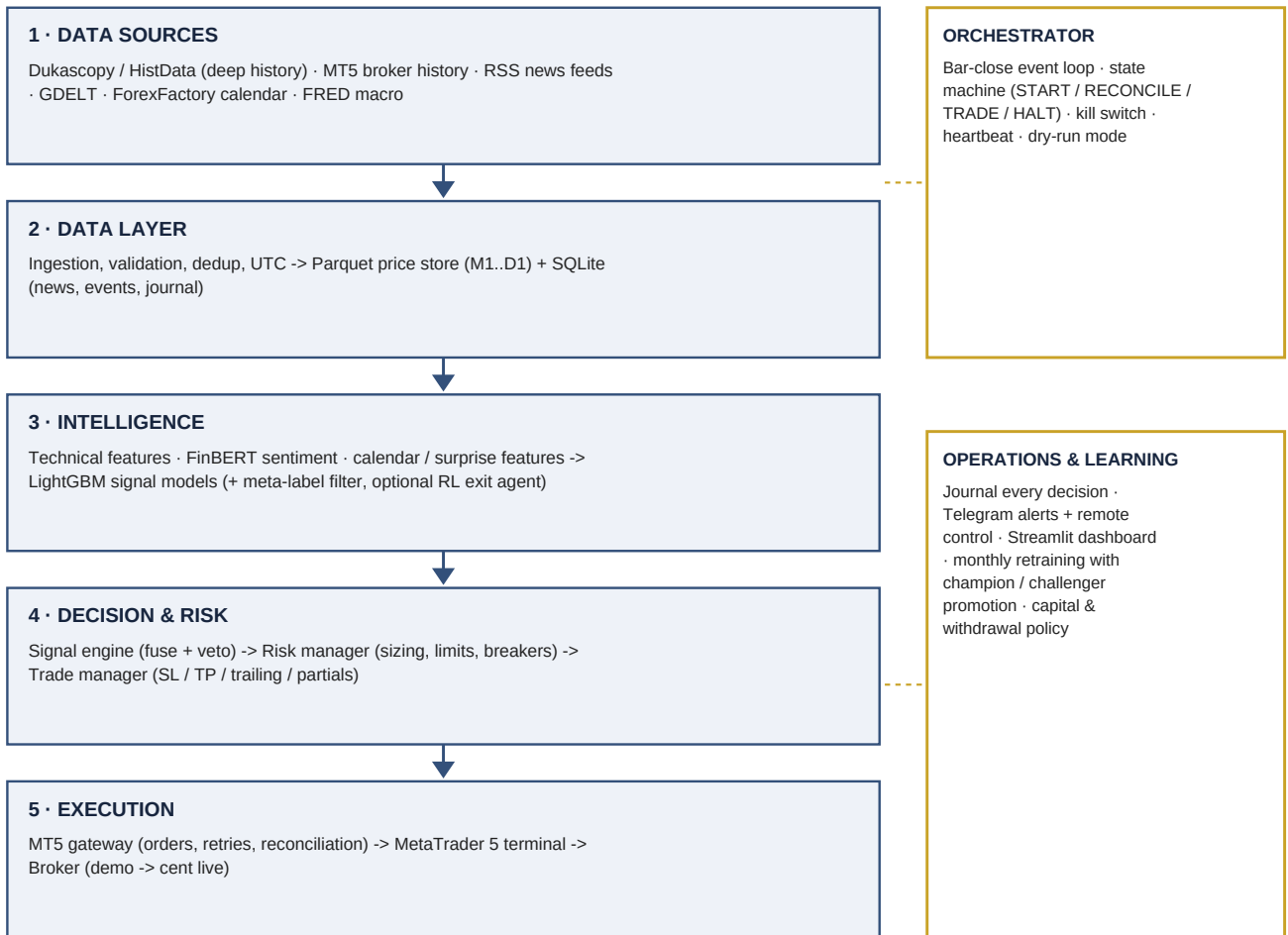
Everything in this plan runs on software that is free for this use: Python, LightGBM, PyTorch, HuggingFace transformers, Stable-Baselines3, SQLite, Parquet, Streamlit, Telegram bots, and the official MetaTrader 5 Python package. Historical price data comes from Dukascopy and HistData (free). News history is the one dataset that is *not* freely available in high quality — Chapter 7 explains how to backfill it from GDELT and archived economic calendars, and how to build your own growing archive from day one, which quickly becomes your proprietary asset. Hosting starts on your own Windows PC (\$0). The only unavoidable ongoing costs are electricity and an internet connection.

2. Guiding Principles

1. **Capital preservation before profit.** Every component is designed so that the worst case is bounded. Hard limits live in the risk manager and cannot be overridden by any model output.
2. **Demo first, cents second, dollars later.** Nothing trades live until it has passed a written go-live checklist on a demo account (Chapter 10).
3. **Free and open-source by default.** Paid services are never load-bearing. Where a free tier is used (e.g. news APIs), the system must degrade gracefully if it disappears.
4. **No lookahead, ever.** Every feature, label and backtest respects information availability at decision time. Leakage is the #1 cause of strategies that shine in backtests and die live.
5. **Everything journaled.** Every decision — including decisions *not* to trade — is logged with the full feature snapshot, so the system can learn from its own history.
6. **Config-driven, instrument-agnostic.** EURUSD, XAUUSD and US100 are configuration entries, not hard-coded logic, so the system generalises to new instruments by adding a config block.
7. **Simple models before complex ones.** Gradient boosting on engineered features is the workhorse; deep RL is an optional refinement, adopted only if it beats the simpler baseline in honest tests.
8. **One prompt per Claude Code session.** Build, review, test, commit — then move to the next prompt. The prompts in Chapter 13 are designed for this cadence.

3. System Architecture

Danalit is a modular monolith: one Python codebase, one machine, clear internal boundaries. This is deliberate — microservices and cloud queues add cost and failure modes that a zero-budget single-operator system does not need. The diagram shows the six layers and the two cross-cutting loops (orchestration and learning).



3.1 Layer responsibilities

Layer	Modules	Responsibility
Data sources	Dukascopy/HistData downloaders, MT5 history, RSS/GDELT collectors, ForexFactory calendar, FRED	Acquire raw prices, news and macro events. All free.
Data layer	price_ingest, news_ingest, calendar_ingest, store	Validate, deduplicate, timezone-normalise (UTC everywhere); Parquet for bars/ticks, SQLite for news, events, trades and decisions.
Intelligence	features.technical, features.sentiment, features.fundamental, labeling, models.train	Engineer leakage-safe features, score news with FinBERT, label with the triple-barrier method, train and evaluate LightGBM models per instrument.
Decision & risk	signal_engine, risk_manager, trade_manager	Fuse model probabilities, sentiment and calendar state into one signal; apply position sizing, exposure and drawdown limits; manage SL/TP/trailing/partial exits.
Execution	mt5_gateway, orchestrator	Talk to the MetaTrader 5 terminal: send/modify/close orders with retries and reconciliation; run the bar-close event loop with a state machine and kill switch.
Operations	journal, monitor.telegram_bot, monitor.dashboard, retrain pipeline	Journal every decision, alert and report via Telegram, visualise via Streamlit, retrain on schedule with champion/challenger promotion.

3.2 The two loops

The fast loop (seconds–minutes): on every completed bar (M15 primary timeframe), the orchestrator pulls fresh bars and news, recomputes features, asks the signal engine for a decision, passes it through the risk manager, and instructs the MT5 gateway. Open positions are managed on every loop tick (trailing stops, breakeven moves, news blackouts, time exits).

The slow loop (weekly–monthly): the journal accumulates every decision with its feature snapshot and outcome. On schedule, the retraining pipeline re-runs walk-forward training on the extended dataset; a challenger model is promoted only if it beats the champion on honest out-of-sample metrics. This is how the system “continuously learns” — through disciplined periodic retraining with promotion gates, not by online-updating weights on every trade (which is how systems destroy themselves).

4. Technology Stack

Every choice below is free for this use case, runs on Windows, has strong community support, and is well known to Claude Code (which matters — it will write better code for mainstream libraries).

Layer	Choice	Why this one
Language	Python 3.11+	The entire ML/trading open-source ecosystem; the official MetaTrader 5 API is a Python package.
Data handling	pandas, numpy, pyarrow	Standard; Parquet via pyarrow gives fast, compact columnar storage for years of M1 bars.
Technical analysis	pandas-ta (plus hand-rolled S/R and patterns)	Pure-Python, no compiler needed on Windows (unlike TA-Lib); covers 130+ indicators.
Core ML	LightGBM + scikit-learn	Gradient boosting is the strongest baseline for tabular financial features; fast to retrain on CPU; interpretable via feature importance.
News NLP	HuggingFace transformers + ProsusAI/finbert	Free finance-tuned sentiment model; scores headlines on CPU in milliseconds; no API cost.
Reinforcement learning	Gymnasium + Stable-Baselines3 (PPO)	Optional exit-management agent; SB3 is the maintained standard for small-scale RL.
Hyperparameter tuning	Optuna	Efficient search with pruning; integrates with LightGBM.
Storage	Parquet files + SQLite (WAL mode)	Zero-admin, zero-cost, single-machine reliable; SQLite comfortably handles news, events, trades, decisions.
Broker connectivity	MetaTrader5 official pip package	Direct, supported API into the MT5 terminal: market data, orders, positions, history.
Scheduling	APScheduler + Windows Task Scheduler	In-process bar-close scheduling; OS level auto-start and watchdog.
Dashboard	Streamlit + plotly	One-file local dashboard; no web hosting needed.
Alerts / remote control	python-telegram-bot	Free push alerts to your phone plus remote status/halt/resume commands.
Testing	pytest	Unit, integration and regression tests throughout.
Versioning	Git (+ optional DVC for datasets)	You already have the repo.

WHY MT5, NOT MT4

MetaTrader 5 has an official Python package (MetaTrader5) that gives your Python process direct access to quotes, bars, order placement and position management while the terminal runs. MT4 has no official API — connecting Python requires a custom EA bridge over ZeroMQ or file exchange, which is fragile and adds weeks of work. Virtually every broker that offers MT4 also offers MT5 demo and cent accounts. Use MT5. (If you are ever forced onto MT4, the accepted fallback is the open-source dwx-zeromq EA bridge — the architecture here still works, only the gateway module changes.)

5. Hosting Strategy

The MetaTrader5 Python package only runs on Windows alongside a running MT5 terminal. That constraint, plus the zero budget, defines the hosting plan:

Phase	Where it runs	Cost	Notes
Development + backtesting	Your Windows PC	\$0	Everything in prompts 1–13 is offline work; no uptime requirement.
Demo forward test	Your Windows PC, or a dedicated old laptop	\$0	The system tolerates downtime by design (state reconciliation on restart, Chapter 11). Missing hours on a demo test is acceptable; note gaps in the journal.
Live (cent account)	Dedicated always-on Windows machine — an old laptop is ideal (built-in battery = free UPS)	\$0	Disable sleep/updates-auto-reboot; auto-start MT5 and Danalit via Task Scheduler; watchdog restarts on crash; Telegram alerts on disconnection.
Later, if profitable	Broker free VPS or cheap Windows VPS	\$0–5/mo	Many brokers (e.g. Exness, RoboForex, FBS) grant a free Windows VPS once equity or monthly volume passes a threshold — that is the natural upgrade path funded by the account itself. Oracle Cloud Free Tier is Linux/ARM and cannot run MT5 sanely; ignore it.

Design consequence baked into the prompts: Danalit must always be safe to kill and restart. On startup it reconciles its database against the broker's actual open positions, adopts or flattens orphans according to config, and resumes. Every position always has a hard SL/TP stored *at the broker*, so even if your machine dies mid-trade, the worst case is bounded.

6. Machine Learning Approach

6.1 The architecture of the intelligence

Danalit deliberately does **not** use one end-to-end deep network that maps raw data to orders. End-to-end deep RL on financial data is notoriously sample-inefficient and overfits catastrophically on the amount of data one operator can gather. Instead the intelligence is layered — each layer simple enough to validate independently:

1. **Signal models (supervised, core).** Per instrument, a LightGBM classifier predicts the outcome of a potential trade using the triple-barrier method: does price hit the profit barrier before the loss barrier within a time horizon? Inputs: ~80–150 engineered technical features (multi-timeframe trend, momentum, volatility, S/R distances, candlestick patterns, session/time encodings) plus fundamental features (sentiment aggregates, calendar proximity, surprise scores). Output: calibrated probabilities for long / short / no-trade.
2. **Meta-label filter (supervised, refinement).** A second model, following López de Prado's meta-labeling idea, learns *when the primary signal is trustworthy* and controls whether to act and how much confidence to assign. This separates “direction” from “conviction” and typically improves precision markedly.

3. **Sentiment engine (pretrained NLP).** FinBERT scores each collected headline (positive/negative/neutral + confidence). Scores are aggregated per currency/asset into rolling features (e.g. net USD sentiment over last 4h/24h) and fed to the signal models — FinBERT itself is never fine-tuned in v1 (not enough labelled data; pretrained is good).
4. **Calendar/event layer (rules).** High-impact events (NFP, CPI, FOMC, ECB) produce hard no-trade blackout windows and post-release “surprise” features (actual vs forecast, z-scored against history). Rules, not ML — this knowledge is reliable and does not need learning.
5. **RL exit agent (optional, last).** A small PPO agent trained in a Gymnasium environment wrapped around the backtester learns only *trade management* (hold / tighten stop / partial close / exit) for already-open positions. Its action space is tiny, so it can actually learn from available data. It is adopted only if it beats the rule-based trade manager out-of-sample.

6.2 Validation doctrine (the part that keeps you honest)

- **Walk-forward everything.** Train on years 1–3, validate on year 4, test on year 5; then roll the window. Final numbers only ever come from data the model never saw in any form.
- **Purging and embargo.** Samples whose label horizon overlaps the validation window are removed (purged) and a gap (embargo) is left after each training window, so label leakage cannot inflate results.
- **Probability calibration.** Raw classifier scores are calibrated (isotonic/Platt) so that “62% confidence” means 62% — position sizing depends on this.
- **Overfitting guards.** Every Optuna search reports the number of configurations tried; the backtester computes deflated Sharpe and a simple probability-of-backtest-overfitting check. A strategy that only shines under one parameter set is discarded.
- **Cost realism.** All training labels and backtests include spread, commission, slippage and swap from day one. An edge that dies under costs is not an edge.

WHAT NOT TO DO (COMMON WAYS THESE PROJECTS DIE)

Do not train on data your live broker cannot give you. Do not tune parameters on the test set — ever, even “just once”. Do not let the model output position size directly (models size positions to insanity; the risk manager sizes positions). Do not online-update model weights from live trades. Do not add instruments before EURUSD alone is stable. Do not trust any backtest whose equity curve you have not inspected trade by trade.

7. Data: Sources and Acquisition

7.1 Historical price data (free)

Source	What you get	How
Dukascopy	Tick data from ~2003: EURUSD, XAUUSD, and USATEC.IDX/USD (NASDAQ 100 CFD). The best free source; institutional-quality Swiss bank feed.	Bulk-download via the open-source dukascopy-node CLI (npm) or direct from their historical data web feed; Prompt 2 builds the downloader.
HistData.com	Free M1 bars for major FX pairs and XAUUSD back to ~2000 (no NASDAQ).	Manual monthly ZIP downloads or scripted; good cross-check against Dukascopy.
Your MT5 terminal	Whatever history your broker serves (often 5–10+ years M1). Crucially, this matches the exact prices/spreads you will trade on.	MetaTrader5 Python package <code>copy_rates_range()</code> ; Prompt 2 ingests it.
Stooq / yfinance	Daily and limited intraday for ^NDX, gold futures — supplementary sanity checks only.	Free HTTP/CSV endpoints; do not build on top of them.

Strategy: use **Dukascopy as the deep training archive** (10+ years, tick-accurate, includes real spreads), and **your broker's MT5 history as the calibration set** — the backtester's cost model is fitted to the broker's actual spreads. Store everything as M1 Parquet and resample upward (M5/M15/H1/H4/D1); never store pre-aggregated data you cannot audit. Expect roughly 3–4M M1 bars per instrument per decade — trivial for Parquet.

7.2 News and economic calendar data

This is the only genuinely hard dataset on a zero budget. Real-time news is easy and free; *timestamped historical* news is what commercial vendors charge thousands for. The plan attacks it three ways: backfill what exists, start archiving immediately (your archive becomes the proprietary asset), and design the models so they degrade gracefully where news history is thin (technical features carry the model; news features enhance it).

Source	What you get	Cost / limits
ForexFactory calendar	The de-facto retail standard economic calendar. A free weekly JSON feed (ff_calendar_thisweek.json on faireconomy.media) gives event, currency, impact, actual/forecast/previous. Historical weeks are browsable and can be scraped politely.	Free. Scrape slowly (1 req/2s), cache everything, respect the site.
FRED (St. Louis Fed)	Authoritative historical macro series (CPI, NFP, rates, PMI...) with an official API — the backbone for historical “surprise” features.	Free API key, generous limits.
GDELT 2.0	Global news metadata every 15 minutes back to 2015, with tone scores and themes; DOC 2.0 API is queryable by keyword (“ECB”, “Fed”, “gold”).	Free, unlimited. Noisy — use aggregates, not single articles.
RSS feeds	Real-time headlines from FXStreet, Investing.com, DailyFX, plus Google News RSS queries per topic. This feeds the live sentiment engine and builds your archive.	Free, poll every 5–10 min.
Finnhub / Marketaux / Alpha Vantage	Optional API-based news with tickers/sentiment on free tiers.	Free tiers are tight (25–100 req/day) — treat as supplements, never load-bearing.

BUILD YOUR OWN NEWS DATASET — STARTING THE WEEK YOU BEGIN

Prompt 3 stands up a collector that polls the calendar feed and RSS sources every few minutes, deduplicates by content hash, and stores everything with exact UTC ingestion timestamps in SQLite. Run it from week 1, even while you are still building the rest. By the time you reach forward testing (roughly month 3–4) you will have months of perfectly timestamped news — exactly aligned with how the live system will see news — which is far more valuable for training the news components than any noisy backfill. GDELT + ForexFactory history + FRED cover the deep past well enough for the calendar features, which are the most predictive fundamental features anyway.

8. MetaTrader Connectivity

The official **MetaTrader5** pip package connects Python directly to a running MT5 terminal on the same Windows machine. It provides: `initialize()/login()`, `symbol_info()` and live ticks, `copy_rates_range()` for bars, `order_send()` for market/pending orders with SL/TP, position and order queries, and full account history. Key facts the design accounts for:

- The MT5 **terminal must be running and logged in**; the gateway module health-checks the connection every loop and auto-reinitialises after terminal restarts.
- Order execution is synchronous with retcodes — the gateway maps every retcode (requote, no money, market closed, trade context busy...) to retry/abort logic with exponential backoff.

- **All SL/TP live at the broker**, attached to the position — never only in Python. Trailing logic modifies the broker-side stop.
- Symbols differ per broker (“XAUUSD” vs “GOLD”, “US100” vs “USTEC” vs “NAS100”): the instruments.yaml config maps Danalit's canonical names to the broker's symbol names, along with contract size, min/step lot, and stops level.
- The same gateway runs against demo and live — only the login config changes, which makes the demo forward test a true dress rehearsal.

9. Risk and Money Management for a \$20 Account

9.1 The arithmetic problem with \$20

On a standard USD account the minimum position is 0.01 lots. For EURUSD that is \$0.10 per pip, so a modest 20-pip stop risks \$2.00 — **10% of a \$20 account on a single trade**. Professional risk is 0.5–1% per trade. The minimum tradable size is 10–20× too large: a standard account with \$20 is structurally untradable, and no software can fix arithmetic.

9.2 The solution: a cent account

Cent accounts (offered free by many MT5 brokers — e.g. RoboForex ProCent, Exness Cent, FBS Cent; availability depends on your country) denominate the balance in cents: your \$20 deposit shows as 2,000¢. The minimum 0.01 cent-lot position on EURUSD is worth about \$0.001 per pip, so a 20-pip stop risks about 2¢ = 0.1% of the account. Suddenly the full professional risk framework fits: you can risk exactly 0.5–1% per trade with sensible stops and scale position size smoothly as equity grows. **Choosing a broker with an MT5 cent account is a hard prerequisite of this plan.** Verify: MT5 (not only MT4), cent account, tight EURUSD spreads, XAUUSD and a NASDAQ 100 CFD available, hedging allowed, no minimum deposit above \$10, and regulation acceptable in your jurisdiction.

9.3 The risk framework (enforced in code, Prompt 10)

Rule	Value (initial)	Purpose
Risk per trade	0.75% of equity	Fixed-fractional sizing from entry-to-SL distance; position size is derived, never chosen by the model.
Max concurrent positions	2 (1 per instrument)	Caps correlation and total heat.
Max total open risk	1.5% of equity	Sum of open-position risk never exceeds this.
Daily loss limit	3% of equity	Trading halts until next day; Telegram alert.
Weekly loss limit	6% of equity	Halts for the week; forces review.
Drawdown circuit breaker	15% from equity high-water mark	System flattens and disables itself until you manually re-enable after review.
News blackout	±15 min around high-impact events on the instrument's currencies	No entries; optionally tighten stops on open positions.
Consecutive-loss brake	4 losses in a row -> risk per trade halves for 24h	Cheap insurance against regime breaks.

9.4 Scaling, profit retention and withdrawals

The capital policy module (Prompt 19) implements equity tiers. Live trading starts with EURUSD only; XAUUSD and US100 unlock as equity grows (they have wider spreads and violent moves that punish tiny accounts). A fixed share of each month's net profit is earmarked to a virtual "set-aside ledger"; when the set-aside balance exceeds the broker's minimum withdrawal it becomes a withdrawal recommendation in the monthly report. Example schedule (all configurable):

Equity	Instruments live	Risk/trade	Profit set-aside
< \$50	EURUSD	0.75%	0% — compound everything
\$50–200	EURUSD + XAUUSD	0.75%	10% of monthly net profit
\$200–1,000	All three	1.0%	25% of monthly net profit
> \$1,000	All three + candidates	1.0%	40% of monthly net profit, monthly withdrawal recommendation

10. Testing Strategy

10.1 The four gates

1. **Unit and integration tests** (continuous): every module ships with pytest tests; the risk manager and backtester get the densest coverage because errors there are unbounded.
2. **Backtesting** (Prompts 8–9): event-driven simulation over 8–12 years, bar-by-bar, with the full cost model. Walk-forward only; minimum ~500 trades per instrument for statistical meaning; inspect equity curves, drawdown profiles, trade duration distributions, and per-regime performance (trending vs ranging, by year).
3. **Monte Carlo stress** (part of Prompt 9's report): bootstrap-resample the trade sequence to get drawdown distributions ("there is a 5% chance of a 22% drawdown"), and re-run with costs $\times 1.5$ and $\times 2$ — the strategy must survive degraded execution.
4. **Forward test on demo** (the only gate that matters): run the complete live system on a demo account for **at least 12 weeks and 100+ trades**. The journal analytics (Prompt 16) compare realised results against backtest expectations trade-by-trade — entry slippage, spread paid, win rate, expectancy. Deviation beyond confidence bands means stop and diagnose.

10.2 Go-live checklist (all must be true)

- ≥ 12 weeks demo forward test, ≥ 100 trades, without manual intervention.
- Forward profit factor ≥ 1.15 and expectancy within the backtest's Monte Carlo bands.
- Max forward drawdown $< 12\%$; daily-loss halt triggered correctly when hit.
- Every operational failure mode rehearsed: terminal restart, PC reboot mid-trade, internet loss, kill switch, weekend gap over an open position.
- Telegram alerting verified end-to-end; dashboard reflects reality against broker statements.
- You have read the runbook (Prompt 20) and can execute every procedure in it.

11. Deployment and Operations

- **Process management:** MT5 terminal and Danalit auto-start at boot via Windows Task Scheduler; a watchdog task restarts Danalit if its heartbeat file goes stale (Prompt 20 builds this). Windows Update deferred; sleep/hibernate disabled.

- **Kill switch:** a HALT file (creatable via Telegram command /halt) stops new entries immediately and optionally flattens; the state machine refuses to trade unless every startup reconciliation check passes.
- **Monitoring:** Telegram bot pushes trade opens/closes, halts, errors, and a daily digest (equity, P&L, open risk, model confidence stats); /status, /positions, /halt, /resume commands work from your phone. Streamlit dashboard shows equity curve vs backtest projection, open positions, recent decisions with explanations, feature drift, and data-feed freshness.
- **Daily routine (5 min):** check digest, confirm data freshness, confirm broker statement matches journal. **Weekly (30 min):** review every trade with its logged explanation, review skipped-signal log, check feature drift report. **Monthly:** retraining gate (Prompt 18), capital policy report (Prompt 19), withdraw set-aside if triggered.

12. Continuous Improvement

The improvement loop is built from three artefacts the system already produces: the journal (every decision + features + outcome), the model registry (every trained model + its walk-forward report), and the drift monitor. On this foundation:

- **Scheduled retraining** (monthly, or triggered by drift): retrain on all data through yesterday, evaluate walk-forward, and promote challenger over champion only if it wins on out-of-sample expectancy *and* drawdown. Automatic rollback if live performance degrades in the first two weeks after promotion.
- **Feature research backlog:** the journal's feature snapshots make offline research cheap — test new features (order-flow proxies, cross-asset signals like DXY/yields for gold, VIX for US100) against recorded decisions before ever touching live code.
- **Instrument expansion:** generalisation = add an instruments.yaml block, backfill data, train, walk-forward, demo-trade the new instrument in shadow mode (signals journaled, not executed) for a month, then enable at the smallest tier.
- **RL expansion:** once the exit agent (Prompt 13) proves itself, the same environment supports researching entry-timing agents — with the same rule: adopted only if it beats the incumbent out-of-sample.

13. The 20 Claude Code Prompts

These prompts are the build plan. Give them to Claude Code **one per session, in order**, inside the Danalit git repository. After each session: read the code Claude wrote, run the tests, run the module by hand once, and commit before starting the next prompt. Prompt 1 creates a CLAUDE.md file that gives every later session the project context automatically. If a session goes sideways, revert to the last commit and re-run the prompt with a note about what went wrong — the prompts are written to be re-runnable.

HOW TO WORK WITH THESE PROMPTS

Each prompt ends with a “Definition of done” that Claude Code should satisfy before finishing — hold it to that. Where a prompt says VERIFY, do the verification yourself; do not take the model's word for it. It is normal and good to follow up within the session (“the test fails with X, fix it”). Avoid mixing prompts in one session: small, reviewed, committed increments beat one giant session every time.

PROMPT 1 OF 20 · PHASE A — FOUNDATION

Prompt 1 — Project scaffold, configuration and database

Creates the repository skeleton, typed configuration, logging, the SQLite schema, and the CLAUDE.md context file that every later session relies on. No trading logic yet.

Paste this into Claude Code:

```
You are starting Danalit: an autonomous AI trading system for MetaTrader 5 that will trade
EURUSD, XAUUSD and US100 using ML signals from technical + news features. This session creates the
project skeleton ONLY - no trading logic.

1. Create this layout:
  danalit/
    (package: data/, features/, models/, backtest/, risk/, trading/, monitor/,
    journal/ subpackages, plus config.py, db.py, logging_setup.py, constants.py)
  config/settings.yaml, config/instruments.yaml
  scripts/ tests/ data_store/ models_store/ reports/
2. config.py: load both YAML files into pydantic models. settings.yaml sections: paths, broker
  (login/server come from env vars DANALIT_MT5_LOGIN/SERVER/PASSWORD - never stored in files), risk
  (risk_per_trade=0.0075, max_positions=2, max_total_risk=0.015, daily_loss_limit=0.03,
  weekly_loss_limit=0.06, max_drawdown=0.15, consecutive_loss_brake=4), trading
  (primary_timeframe=M15,
  loop_interval_sec=30), news (poll_minutes=5). instruments.yaml: one block per instrument with
  canonical name, broker_symbol, pip_size, min_lot, lot_step, spread_estimate_pips, sessions,
  enabled flag, and news_currencies (e.g. XAUUSD -> [USD], US100 -> [USD]).
3. logging_setup.py: rotating file handler per component under logs/, plus console; UTC timestamps.
4. db.py: SQLite at data_store/danalit.db, WAL mode, with schema + migrations for tables: news(id,
  source, published_utc, ingested_utc, title, body, url, content_hash UNIQUE), calendar_events(id,
  source, event_utc, currency, name, impact, actual, forecast, previous, revised),
  trades, decisions, equity_snapshots, model_registry. Add `python -m danalit.db --init`.
5. requirements.txt: pandas, numpy, pyarrow, pydantic, PyYAML, requests, beautifulsoup4, feedparser,
  pandas-ta, lightgbm, scikit-learn, optuna, transformers, torch, stable-baselines3, gymnasium,
  MetaTrader5, APScheduler, streamlit, plotly, python-telegram-bot, pytest. Pin major versions.
6. Write CLAUDE.md: project purpose, architecture summary, layout, conventions (ALL timestamps
  timezone-aware UTC; no lookahead in any feature; risk limits are inviolable and live only in
  danalit/risk/; config-driven instruments; every module gets pytest tests), how to run tests.
7. tests: config loads and validates; db --init creates all tables; a second --init is a no-op.
Definition of done: pytest passes; `python -m danalit.db --init` works on Windows; CLAUDE.md and
README.md exist and are accurate. Do not implement any other module yet.
```

PROMPT 2 OF 20 · PHASE A — FOUNDATION

Prompt 2 — Historical price data pipeline

Downloads a decade of free historical data from Dukascopy, ingests the broker's own MT5 history, validates and stores everything as M1 Parquet with resampling to higher timeframes.

Paste this into Claude Code:

Read CLAUDE.md first. Build the historical price data pipeline in danalit/data/.

1. price_store.py: Parquet-backed store under data_store/prices/{instrument}/{timeframe}/, one file per year. API: write_bars(df), read_bars(instrument, timeframe, start, end), resample(instrument, from_tf, to_tf) producing M5/M15/H1/H4/D1 from M1 (OHLCV aggregation, bars labelled by OPEN time, UTC). Columns: time_utc, open, high, low, close, tick_volume, spread.
2. dukascopy_ingest.py: ingest CSVs produced by the open-source `dukascopy-node` CLI (document the exact npx command in the module docstring for EURUSD, XAUUSD and USATECIDXUSD, M1, 2014->today). Parse, validate, convert to our schema. Do not reimplement the downloader - wrap the CLI's output.
3. mt5_history_ingest.py: pull M1 bars from the local MT5 terminal via the MetaTrader5 package (copy_rates_range in chunks) for the three broker symbols in instruments.yaml. Handle: terminal not running (clear error), missing history (fetch in chunks going back until empty), broker timezone -> UTC conversion (read the server offset from terminal info, document the approach).
4. quality.py: data quality report per instrument - gaps vs expected trading calendar, zero-volume bars, duplicate timestamps, OHLC sanity (high>=low etc.), weekend data, spread outliers. Output a markdown report to reports/data_quality_{instrument}.md.
5. scripts/build_price_data.py: end-to-end CLI - ingest available Dukascopy CSVs + MT5 history, merge (Dukascopy as base, broker data preferred for the overlap period - keep a source column), write M1 Parquet, build all resampled timeframes, run quality report.
6. tests: resampling correctness on synthetic data (known M1 -> known M15), gap detection, dedup, UTC handling around DST.

Definition of done: pytest passes; running scripts/build_price_data.py with a sample CSV I provide (or a generated synthetic one) produces Parquet + a quality report. Print clear instructions for the one-time dukascopy-node download I must run myself.

PROMPT 3 OF 20 · PHASE A — FOUNDATION

Prompt 3 — News and economic-calendar collector

Stands up the always-on collector for the economic calendar and news headlines, plus one-time backfill scripts for FRED and GDELT. Run it 24/7 from this week on — the archive it builds becomes your proprietary training data.

Paste this into Claude Code:

Read CLAUDE.md first. Build the news/calendar layer in danalit/data/. Everything stores into the SQLite tables from Prompt 1, deduplicated by content_hash, with both published_utc and ingested_utc so we always know when the system COULD have known something.

1. calendar_ingest.py: fetch the ForexFactory weekly calendar JSON feed (https://nfs.faireconomy.media/ff_calendar_thisweek.json) -> calendar_events rows (event_utc, currency, name, impact, actual/forecast/previous). Idempotent upserts; re-fetch updates the 'actual' values after releases. Add a polite scraper for past weeks of the ForexFactory calendar (1 request per 2s, on-disk HTTP cache, resumable) to backfill 2014->today; follow robots/ToS and make the rate limit configurable.
2. news_ingest.py: poll RSS feeds (feedparser) from a configurable list - default: FXStreet news, Investing.com news RSS, DailyFX, plus Google News RSS queries for: "federal reserve", "ECB", "gold price", "nasdaq". Store title, summary, source, url, published_utc, ingested_utc.
3. fred_ingest.py: given a FRED API key (env var FRED_API_KEY, free), backfill key US/EU series with release dates: CPI, core CPI, NFP, unemployment, fed funds, PCE, GDP, retail sales, plus ECB rate. Store as calendar_events with source='fred' where they represent releases.
4. gdel_t_backfill.py: one-time script using the GDELT DOC 2.0 API to pull historical article counts and average tone per day for keyword sets (fed/ECB/gold/nasdaq themes), 2015->today, into a new gdel_t_daily table. Rate-limit politely; resumable.
5. collector_daemon.py: APScheduler process that runs calendar fetch every 30 min and RSS polls every 5 min, forever, with per-source error isolation (one dead feed must not stop others), logging, and a heartbeat file. Add scripts/run_collector.py and a Windows Task Scheduler setup note in README.
6. tests: dedup on identical content; calendar upsert updates 'actual' without duplicating; RSS parse of a saved fixture file.

Definition of done: pytest passes; collector daemon runs and populates the DB from live feeds; backfill scripts are resumable after Ctrl+C. Show me row counts per table after a 10-minute run.

PROMPT 4 OF 20 · PHASE B — FEATURES & LABELS

Prompt 4 — Technical feature engineering

The leakage-safe technical feature engine: indicators, candlestick patterns, support/resistance distances, multi-timeframe context and session encodings.

Paste this into Claude Code:

Read CLAUDE.md first. Build danaliti/features/technical.py + tests. This module turns bar data into the model's technical feature matrix. Correctness rule that overrides everything: a feature value at bar T may use ONLY data with time_utc <= T's bar CLOSE time; features are consumed at the next bar's open. Structure every computation so this holds, and write a test that proves it (compute features on full history, then on history truncated at T, values at T must be identical).

Features, computed on the primary timeframe (M15) unless noted, all config-driven:

1. Trend/momentum: EMA(8/21/50/200) distances (in ATR units) and slopes; MACD histogram + slope; RSI(14) + divergence flag; ADX(14); rate-of-change over 4/16/96 bars.
2. Volatility: ATR(14) normalized by price; ATR percentile over 90 days; Bollinger width and %B; high-low range regime flag.
3. Structure: distance (ATR units) to rolling swing highs/lows (fractal pivots over 20/100/500 bars); round-number proximity (00/50 levels for EURUSD, 10s for XAUUSD, 100s for US100); yesterday/last-week high-low distances.
4. Candlestick patterns via pandas-ta cdl functions (engulfing, doji, hammer, star, etc.) collapsed into bullish/bearish/none pattern scores.
5. Multi-timeframe context: H1 and H4 trend direction (EMA stack), H4 RSI, D1 ATR percentile - computed on those timeframes then joined onto M15 rows AS OF the last CLOSED higher-TF bar (no peeking into a forming H4 bar; test this explicitly).
6. Time encodings: sin/cos of hour-of-day and day-of-week; session flags (Tokyo/London/NY/overlap); minutes-since-session-open; bars-to-weekend.

API: build_features(instrument, start, end) -> DataFrame indexed by time_utc, plus a FEATURE_REGISTRY dict with name -> description/params for documentation and drift monitoring later. NaN policy: warmup rows dropped; no forward-filling across weekends.

Definition of done: pytest passes including the no-lookahead truncation test and the higher-TF as-of-join test; building 5 years of EURUSD M15 features completes in under ~2 minutes and produces a documented feature matrix with zero NaNs.

PROMPT 5 OF 20 · PHASE B — FEATURES & LABELS

Prompt 5 — News sentiment and fundamental features

FinBERT sentiment scoring of the news archive, calendar proximity/surprise features, and their leakage-safe alignment onto the price timeline.

Paste this into Claude Code:

Read CLAUDE.md first. Build danaliti/features/sentiment.py and danaliti/features/fundamental.py.

1. sentiment.py: score every news row in SQLite with ProsusAI/finbert (HuggingFace transformers, CPU): P(positive), P(negative), P(neutral) on the title (+ summary if present). Batch, cache scores back into a news_scores table keyed by news id + model version so re-runs are incremental. Add a simple keyword->entity tagger mapping each headline to affected assets: currencies (USD, EUR), gold, nasdaq/equities - keyword lists in config, overlappable.
2. Sentiment features (aligned to M15 bars, using ingested_utc NOT published_utc - we can only act on news when we receive it; this is the leakage rule for news): per instrument, rolling net sentiment (pos-neg weighted by confidence) over 1h/4h/24h windows for each relevant entity; article count vs 30-day average (news intensity); max single-article negativity in last 4h.
3. fundamental.py - calendar features per M15 bar per instrument, from calendar_events: minutes_to_next_high_impact (capped), minutes_since_last_high_impact, count of high/medium events next 24h for the instrument's news_currencies, blackout flag (within +/-15min of high impact), and post-release surprise features: (actual - forecast) z-scored against that event name's own history, decayed over 8h. Event-name normalization table so 'Non-Farm Employment Change' variants map to one canonical event.
4. Where history is thin (before your collector started), features must degrade to neutral values with an availability mask column per feature group, so models can learn with and without news.
5. Extend build_features() (Prompt 4) with a flag to include these; update FEATURE_REGISTRY.
6. tests: alignment correctness (a headline ingested at 12:07 first affects the bar closing 12:15); surprise z-score on synthetic history; blackout flag edges; neutral-degradation masks.

Definition of done: pytest passes; scoring the archive is incremental and resumable; building the combined technical+news feature matrix for 1 year of EURUSD works and documents every feature.

PROMPT 6 OF 20 · PHASE B — FEATURES & LABELS

Prompt 6 — Labeling and dataset builder

Triple-barrier labels, purged walk-forward splits, and versioned training datasets — the foundation of honest ML.

Paste this into Claude Code:

Read CLAUDE.md first. Build `danalit/features/labeling.py` and `danalit/features/dataset.py`.

1. `labeling.py` - triple-barrier method: for each bar T (potential entry at next bar open), set an upper barrier at $\text{entry} + k_{tp} * \text{ATR}(14)$, lower at $\text{entry} - k_{sl} * \text{ATR}(14)$, and a time barrier of N bars (defaults $k_{tp}=2.0$, $k_{sl}=1.0$, $N=96$ on M15; all configurable). Walk forward bar by bar: label +1 if upper hit first, -1 if lower hit first, 0 on time-out (use the sign of return at timeout with a dead zone of $\pm 0.25 * \text{ATR} \rightarrow \text{else } 0$). CRITICAL COSTS: entry price includes the instrument's spread_estimate (buy at ask), barrier checks use bid for longs. Also emit: the exact barrier prices, hit time, and the realized return - the backtester will reuse these definitions. Build labels for BOTH direction cases (long-framed and short-framed) so we can train a 3-class problem: profitable-long / profitable-short / no-trade.
2. `dataset.py`: assemble (features from Prompts 4-5) + labels into training datasets. Implement purged walk-forward splitting: given fold boundaries (e.g. train 2016-2021, validate 2022, test 2023-2024, then roll), PURGE training samples whose label window overlaps the validation period and add a configurable EMBARGO (default 5 days) after each train window. No random shuffles anywhere. Persist datasets under `data_store/datasets/{instrument}/{dataset_version}/` with a manifest.json (feature list, label params, fold boundaries, row counts, class balance, git commit).
3. `scripts/build_dataset.py` CLI: instrument + config $\rightarrow$ dataset on disk + printed summary table of fold sizes and class balance.
4. tests: barrier-hit correctness on hand-constructed price paths (including gap-through-barrier); purge/embargo correctness on synthetic overlapping windows; determinism (same inputs $\rightarrow$ identical dataset hash).

Definition of done: pytest passes; datasets build for all three instruments; class balance and fold summaries look sane (no fold with <500 samples; document any that are).

PROMPT 7 OF 20 · PHASE C — MODELS

Prompt 7 — Baseline signal models

LightGBM classifiers per instrument with probability calibration and a full evaluation report — the honest first look at whether there is an edge.

Paste this into Claude Code:

Read CLAUDE.md first. Build `danalit/models/train.py`, `calibrate.py`, `evaluate.py`, `registry.py`.

1. `train.py`: LightGBM multiclass (long/short/no-trade) per instrument on the Prompt 6 datasets. Sensible defaults (max_depth 4-6, min_child_samples high, feature_fraction ~ 0.8 , early stopping on the validation fold); class weights for imbalance. Train per walk-forward fold; the model for fold k never sees fold k 's validation/test data.
2. `calibrate.py`: isotonic calibration of predicted probabilities on each fold's validation split; reliability diagrams saved to reports/.
3. `evaluate.py` $\rightarrow$ reports/model_eval_{instrument}_{version}.md with, per fold and pooled out-of-sample: log loss, per-class precision/recall, and the TRADING-RELEVANT metrics - for thresholded decisions (act when $\max(P_{\text{long}}, P_{\text{short}}) > \tau$ for τ in $0.45..0.70$): number of signals, hit rate, average labeled return per signal AFTER the label's cost model, and expectancy. Plot expectancy vs τ . Also: top-30 feature importances (gain), grouped technical vs news vs calendar so we can see what carries the model; performance by year and by session.
4. `registry.py`: every trained model saved to `models_store/{instrument}/{version}/` with model files per fold, calibrators, feature list, dataset version, metrics.json, git commit; registry table from Prompt 1 updated. `load_latest(instrument)` API for later modules.
5. `scripts/train_models.py` CLI for all instruments.
6. tests: training on a tiny synthetic dataset with a planted pattern recovers it; registry round-trip; calibration output monotonic.

Definition of done: pytest passes; full train+eval runs for EURUSD end-to-end; the eval report renders and is comprehensible. IMPORTANT: report results honestly - if out-of-sample expectancy is negative or near zero, say so plainly in the report summary; do not tune on test data to fix it.

PROMPT 8 OF 20 · PHASE C — MODELS

Prompt 8 — Event-driven backtesting engine

A bar-by-bar backtester with full cost modelling whose trade lifecycle logic is the same logic the live system will use.

Paste this into Claude Code:

Read CLAUDE.md first. Build `danalit/backtest/`: `engine.py`, `costs.py`, `metrics.py`, `report.py`. Design constraint: the backtester consumes a Strategy interface (`on_bar(context) -> orders`) and a cost model, and its position/SL/TP lifecycle semantics must EXACTLY match how MT5 handles them, because Prompts 11-15 reuse these semantics live.

- `engine.py`: event-driven loop over M15 bars (multi-instrument aware, one portfolio). Orders created at bar close execute at NEXT bar open +/- slippage. Intrabar SL/TP resolution using the bar's OHLC with the pessimistic rule (if both SL and TP lie within a bar's range, assume SL hit first). Gap handling: if price gaps past SL/TP, fill at the gapped open, not the stop price. Support: market orders, SL/TP attached, modify-SL/TP (for trailing), partial close, close-all. Track equity/balance/margin per bar; margin model from instrument contract specs and configurable leverage (default 1:500 cent-account style).
- `costs.py`: per instrument - spread (default: per-bar recorded spread if present in data, else config estimate; entries pay it), commission per lot, slippage model (configurable: fixed pips + extra during the 15 min around high-impact calendar events), and swap per night held including triple-swap Wednesday. All parameters in config with our broker's real values to be filled in.
- `metrics.py`: net profit, profit factor, expectancy, win rate, max drawdown (both equity and balance basis), CAGR, Sharpe and Sortino (bar returns), trade duration stats, consecutive loss runs, per-instrument and per-year breakdowns.
- `report.py`: HTML report (plotly) - equity curve with drawdown shading, monthly returns table, trade distribution charts, cost breakdown (gross vs net - show exactly how much spread/swap ate).
- Include two toy strategies for validation: buy-and-hold, and MA-cross.
- tests: known synthetic price path + scripted strategy -> hand-computed P&L matches to the cent; pessimistic SL/TP rule; gap fills; swap accrual days; partial closes.

Definition of done: pytest passes; MA-cross backtest over 5 years of real EURUSD data runs in seconds-to-minutes and the HTML report opens with sane numbers (buy-and-hold P&L within rounding of a hand calculation).

PROMPT 9 OF 20 · PHASE C — MODELS

Prompt 9 — Walk-forward harness and honest optimisation

Connects models to the backtester, tunes hyperparameters with Optuna inside each fold only, and guards against overfitting with deflated Sharpe and Monte Carlo stress tests.

Paste this into Claude Code:

Read CLAUDE.md first. Build `danalit/backtest/walkforward.py` and `danalit/models/tuning.py`.

- `walkforward.py`: orchestrate the full honest pipeline per instrument: for each walk-forward fold - train (Prompt 7) on train split, tune on validation split ONLY, then run the Prompt 8 backtester over the fold's TEST period using an `MLSignalStrategy` that: computes features bar by bar, asks the fold's model for calibrated probabilities, enters when $\max(P_{\text{long}}, P_{\text{short}}) > \tau$ with SL/TP at the label-definition barriers ($k_{\text{sl}}, k_{\text{tp}} \times \text{ATR}$), sized by a placeholder fixed-fractional `sizer` (the real risk manager arrives in Prompt 10). Stitch fold test periods into one continuous out-of-sample equity curve - this is THE headline result of the whole project so far.
- `tuning.py`: Optuna over ($\tau, k_{\text{tp}}, k_{\text{sl}}, N$ horizon, LightGBM depth/leaves/min_child) maximizing validation expectancy net of costs, with pruning. HARD RULES: objective only ever evaluates on the fold's validation split; test periods are touched exactly once, by the final configuration; log every trial to SQLite so the search is resumable and auditable.
- Overfitting guards in the final report: deflated Sharpe ratio given the number of trials; parameter-sensitivity table (performance of $\tau \pm 0.05$, k 's $\pm 25\%$ - a real edge degrades smoothly, a fake one cliffs); Monte Carlo bootstrap of trade sequence -> distribution of max drawdown and $P(\text{ruin})$ at the Prompt-10 risk settings; cost-stress rerun at 1.5x and 2x costs.
- `scripts/run_walkforward.py` CLI -> reports/walkforward_{instrument}_{version}.html combining everything, with a plain-language verdict section template: edge or no edge, and confidence.
- tests: fold isolation (assert no test-period timestamps ever reach training/tuning code paths - instrument this with an explicit guard class, not just discipline); resumability.

Definition of done: pytest passes; full walk-forward for EURUSD completes and produces the report. Report honestly. If expectancy net of costs is negative, the verdict section must say so and list the Chapter-14 fallback options (feature enrichment, H1 timeframe, session filters, meta-labeling over a rule-based signal) rather than suggesting we tune until it looks good.

PROMPT 10 OF 20 · PHASE D — DECISION & RISK

Prompt 10 — Risk manager

The inviolable risk core: position sizing, exposure caps, loss limits and circuit breakers, engineered for cent-account arithmetic.

Paste this into Claude Code:

Read CLAUDE.md first. Build danalit/risk/risk_manager.py + position_sizing.py. This module is the most safety-critical code in the project: every order the system ever sends passes through it, in backtest and live identically. No other module may compute a position size.

1. position_sizing.py: fixed-fractional sizing - $\text{size} = (\text{equity} * \text{risk_per_trade}) / (\text{sl_distance in price units} * \text{value per price unit per lot})$, then FLOOR to the instrument's lot_step, respect min_lot and max_lot, and respect margin headroom at configured leverage. Cent-account aware: all arithmetic in account currency units from config (USD cents), with tests proving a \$20 (2000 cent) account sizes EURUSD trades at ~0.75% risk with 15-40 pip stops without breaching min_lot. If min_lot forces risk above a configurable hard cap ($1.5x \text{risk_per_trade}$), REFUSE the trade and return the reason.
2. risk_manager.py: stateful gatekeeper with API check_order(proposed) -> Approved(sizing) | Rejected(reason), plus on_trade_closed(), on_equity_snapshot(). Enforce, from config: max concurrent positions (total and per instrument), max total open risk, daily loss limit (UTC day, realized + open P&L), weekly loss limit, max drawdown from equity high-water mark -> FLATTEN_AND_HALT state requiring manual reset via a CLI command, consecutive-loss brake (after 4 straight losses halve risk for 24h), and instrument enablement by equity tier (equity tiers from config: <\$50 EURUSD only, etc. - Chapter 9 table). State persists to SQLite so restarts cannot reset daily/weekly counters or the high-water mark.
3. Every rejection is logged with full context (this feeds the journal in Prompt 16).
4. Wire it into the backtester as the sizer, replacing Prompt 9's placeholder; add a config flag to re-run walk-forward with full risk management to see realistic small-account results.
5. tests: dense unit coverage - sizing math to the cent for all three instruments, every limit trigger and reset boundary (day rollover, week rollover), HWM persistence across simulated restart, min_lot refusal, tier gating.

Definition of done: pytest passes with high coverage on this module (aim >95% branch coverage); walk-forward rerun with the risk manager produces a report; sizing behaves correctly for a 2000-cent account in a documented worked example.

PROMPT 11 OF 20 · PHASE D — DECISION & RISK

Prompt 11 — Trade management

Rule-based management of open positions: initial SL/TP placement, breakeven moves, ATR trailing, partial take-profit, time exits, and news protection.

Paste this into Claude Code:

Read CLAUDE.md first. Build danalit/trading/trade_manager.py. It manages OPEN positions only (entries are the signal engine's job, Prompt 12). It must run identically inside the backtester (called each bar) and live (called each loop tick), operating on a normalized Position object.

Rules, all config-per-instrument:

1. Initial protection: every entry gets $\text{SL} = k_{\text{sl}} * \text{ATR}$ and $\text{TP} = k_{\text{tp}} * \text{ATR}$ from the label definition (Prompt 6) - consistency between training labels and live behavior is the point.
2. Breakeven: when unrealized profit $\geq 1.0 * \text{ATR}$, move SL to entry + spread (locks 0 minus costs).
3. ATR trailing: after breakeven, trail SL at $1.5 * \text{ATR}$ behind the best price achieved, monotonic only (never widen).
4. Partial TP: close 50% at $+1.0 * \text{ATR}$ (configurable), let the rest run to TP/trail. Respect min_lot: if the position cannot be split ($\text{size} < 2 * \text{min_lot}$), skip partials.
5. Time exit: close any position older than N_hold bars (default = label horizon N from Prompt 6) at market.
6. News protection: if a high-impact event for the instrument's currencies is within the blackout window (Prompt 5's calendar features), either tighten SL to $0.5 * \text{ATR}$ or flatten, per config.
7. Weekend rule: flatten all positions Friday at a configured UTC time (default 20:30) unless config says carry; never carry cent-account XAUUSD over weekends.

Emit a management action log (position, rule fired, before/after) for the journal.

Wire into the backtester and rerun EURUSD walk-forward with management on vs off; add the comparison to the report (management should reduce drawdown; expectancy may drop slightly - both outcomes are acceptable, we need the data).

tests: each rule on scripted price paths; monotonic trailing; min_lot partial skip; interaction ordering (breakeven before trailing; news rule overrides others; time exit last).

Definition of done: pytest passes; on-vs-off comparison report exists; rule ordering is documented in the module docstring.

PROMPT 12 OF 20 · PHASE D — DECISION & RISK

Prompt 12 — Unified signal / decision engine

Fuses model probabilities, sentiment, calendar state and regime filters into one explainable trading decision — the brain that the orchestrator will call every bar.

Paste this into Claude Code:

```
Read CLAUDE.md first. Build danalit/trading/signal_engine.py - the single decision point.
API: decide(instrument, now) -> Decision(action: LONG|SHORT|NONE, confidence: float, sl_price,
tp_price, explanation: str, features_snapshot: dict). It must be pure/deterministic given its
inputs (no side effects) so it is identically testable in backtest and live.

Decision pipeline, in order (each stage can veto, vetoes are recorded with reasons):
1. Data freshness gate: bars current to within 2 bar-intervals; news collector heartbeat fresh.
   Stale data -> NONE("stale data").
2. Calendar gate: inside a blackout window -> NONE("news blackout").
3. Regime filter: config-driven - default: require ADX(H1) > threshold OR ATR percentile in a
   tradable band (from walk-forward analysis of where the model actually earns; expose as config).
4. Model signal: load champion model (registry, Prompt 7), compute calibrated P_long/P_short/P_none
   from the current feature vector; require max(P_long, P_short) > tau (per-instrument tau from the
   walk-forward result).
5. Sentiment veto (config-gated): block LONG if strongly negative net 4h sentiment for the
   instrument (beyond a configured percentile), and vice versa. Log vetoes distinctly - after 3
   months of journal data we will evaluate whether this veto earns its keep.
6. Confluence bonus: if model direction agrees with H4 trend direction AND sentiment sign,
   confidence = base + bonus (affects journal analytics only, not sizing, in v1).
Output SL/TP prices from the ATR barriers. explanation is a one-line human-readable summary,
e.g.: "LONG EURUSD p=0.64 tau=0.55 | H4 up | sent +0.3 | ADX 28 | next high-impact 4h12m".
Every call - including NONE decisions - returns the full features_snapshot for journaling.
Replace the walk-forward MLSignalStrategy internals with this engine (one code path everywhere).
tests: each veto stage with synthetic inputs; determinism; explanation formatting; a golden-file
test of decisions over a fixed historical week so refactors can't silently change behavior.
Definition of done: pytest passes; walk-forward rerun through the unified engine reproduces prior
results within tolerance; decisions carry complete explanations and snapshots.
```

PROMPT 13 OF 20 · PHASE D — DECISION & RISK

Prompt 13 — Optional RL exit agent

A tightly-scoped PPO agent that learns trade management, evaluated against the rule-based manager — adopted only if it wins out-of-sample. Skippable without breaking anything.

Paste this into Claude Code:

```
Read CLAUDE.md first. This prompt is OPTIONAL and must not block the rest of the system -
build it as a pluggable alternative to the Prompt 11 trade manager, selected by config.

1. danalit/models/rl_exit/env.py: a Gymnasium environment wrapping the Prompt 8 backtester for
   SINGLE open-position management. Episode = one historical trade entered by the Prompt 12 engine
   (sample entries from walk-forward TRAINING folds only). Observation: normalized unrealized P&L
   in ATR units, bars-in-trade / horizon, current ATR percentile, distance to SL and TP in ATR,
   H1 trend agreement flag, minutes to next high-impact event (capped/normalized), position
   fraction remaining. Action space (Discrete 4): hold / tighten SL to 1.0*ATR behind price /
   close 50% / close all. Reward: realized P&L change per step net of costs, small time penalty.
2. rl_exit/train.py: PPO via stable-baselines3, CPU-friendly (small MLP), seeded, with eval
   callback on validation-fold trades; save under models_store/rl_exit/{version}/. Log to SQLite.
3. rl_exit/policy_manager.py: a TradeManager-compatible adapter so the backtester and live loop can
   swap it in via config.
4. Evaluation (the only thing that matters): run walk-forward TEST folds twice - rule-based manager
   vs RL manager, same entries, same costs. Compare expectancy, max drawdown, average trade
   duration. Write reports/rl_exit_eval.html with a plain verdict: ADOPT only if it improves
   expectancy without worsening drawdown materially; otherwise KEEP RULES and record why.
5. tests: env step/reset contracts, action effects on the simulated position, reward accounting vs
   hand-computed episode, adapter parity (NOOP policy == rule-manager-with-rules-disabled).
Definition of done: pytest passes; training runs for a small number of steps as a smoke test; the
eval report renders with an explicit ADOPT/KEEP-RULES verdict. If sample efficiency is poor (likely
with limited trades), say so - shipping 'KEEP RULES' is a fully successful outcome of this prompt.
```

PROMPT 14 OF 20 · PHASE E — EXECUTION

Prompt 14 — MT5 broker gateway

The only module that talks to MetaTrader 5: connection management, symbol mapping, robust order operations with retcode handling, and position reconciliation.

Paste this into Claude Code:

```
Read CLAUDE.md first. Build danalit/trading/mt5_gateway.py - the ONLY module allowed to
import MetaTrader5. Everything else uses its normalized types (Position, OrderResult, Tick, Bar).
```

1. Connection: initialize()/login() from env-var credentials; ensure_connected() health check used before every operation, with automatic re-initialize on terminal restart; clear fatal error if the terminal is not installed/running. All prices/times normalized to UTC + Decimal-safe floats.
2. Symbol layer: resolve canonical instrument -> broker symbol from instruments.yaml; on startup verify each symbol exists, is visible (symbol_select), and cache contract specs (contract size, tick value/size, min/step/max lot, stops_level, freeze_level, trade mode). VALIDATE config against live specs and refuse to start on mismatch (e.g. our min_lot config wrong).
3. Orders: market_order(instrument, side, lots, sl, tp, comment) using order_send with correct filling mode detection (IOC/FOK per symbol); modify_position_sltp(); close_position() and partial close by ticket; every operation returns a normalized OrderResult with the raw retcode. Retcode policy table: requote/price-off -> retry up to 3x with fresh price and small backoff; no-money/invalid-stops/market-closed -> fail fast with typed errors; trade-context-busy -> short backoff retry. Respect stops_level minimum SL/TP distances by adjusting or refusing (log which).
4. State: get_open_positions() (filtered to our magic number), get_account() (balance, equity, margin), get_deals_history(since) for reconciliation. All our orders tagged with a config magic number + structured comment (danalit:{signal_id}).
5. reconcile(journal_state) -> report of positions in broker-not-journal (orphans) and journal-not-broker (ghosts); policy hooks: adopt orphan / flatten orphan / mark ghost closed.
6. scripts/gateway_smoke_test.py: against a DEMO account - connect, validate symbols, open 0.01 EURUSD with SL/TP, modify SL, partial close if lot allows, close, print deals. Big red warning in the script if the account is not demo (account_info trade_mode check) - abort unless --i-know-this-is-live.
7. tests: pure-logic tests with a mocked MetaTrader5 module (retcode policy, symbol validation, reconciliation cases); the smoke test is the integration test, run manually.

Definition of done: pytest passes without a terminal; smoke test runs clean against my demo account (I will run it and paste output); reconciliation report correct in mocked scenarios.

PROMPT 15 OF 20 · PHASE E — EXECUTION

Prompt 15 — Live orchestrator

The main loop: a state machine that ties data, signals, risk, trade management and the gateway into a safe, restartable live process with a kill switch.

Paste this into Claude Code:

Read CLAUDE.md first. Build danalit/trading/orchestrator.py + scripts/run_trading.py. This is the live entry point. Design it as an explicit state machine: STARTING -> RECONCILING -> TRADING -> HALTED (recoverable) -> STOPPED, with transitions logged and visible.

1. Startup sequence (any failure -> HALTED with reason, never trade on a broken start): load config + champion models; gateway connect + symbol validation; risk manager state load (HWM, daily/weekly counters); reconcile positions vs journal - apply configured orphan/ghost policy and log every resolution; verify data freshness (price store current, collector heartbeat).
 2. Main loop (APScheduler): every loop_interval_sec: (a) refresh bars from the terminal into the price store, refresh calendar/news state; (b) on a NEW COMPLETED primary-timeframe bar per instrument: run trade_manager on open positions first, then signal_engine.decide(), then risk_manager.check_order(), then gateway execution; (c) every tick: check kill switch and risk halts; (d) heartbeat file + equity snapshot each loop.
 3. Kill switch: if file HALT exists in repo root (or Telegram /halt later): no new entries; HALT_FLAT additionally flattens everything. Deleting the file does NOT auto-resume; resume is an explicit CLI/Telegram action to prevent flapping.
 4. Dry-run mode (config): full pipeline but gateway calls are logged-not-sent, decisions journaled with mode='dry_run'. This is our first end-to-end test and the default until changed.
 5. Crash safety: every intended order is journaled BEFORE sending with a client-side id; on restart reconciliation matches intents to broker deals so a crash between send and ack cannot double-fire.
Global exception handler: log, Telegram-notify (hook exists from Prompt 17, stub for now), transition to HALTED - never a silent dead process (heartbeat watchdog covers hangs, Prompt 20).
 6. tests: state machine transitions with mocked components; the two-generals crash window (kill the process between order-journal and 'ack' in a mock, restart, assert no duplicate order); new-bar detection incl. weekend gaps; dry-run never touches the gateway send path.
- Definition of done: pytest passes; run_trading.py runs in dry-run against the live demo terminal data feed for an hour, journaling decisions every M15 bar with zero unhandled exceptions (I will run this). Document the runbook basics in README.

PROMPT 16 OF 20 · PHASE E — EXECUTION

Prompt 16 — Trade journal and forward-test analytics

Every decision, order and outcome recorded with full context, plus the analytics that compare forward-test reality against backtest expectations — the evidence base for go-live.

Paste this into Claude Code:

Read CLAUDE.md first. Build danalit/journal/journal.py + analytics.py. The journal already receives writes from earlier modules; this prompt completes and hardens it.

1. journal.py: finalize schemas + APIs for: decisions (every decide() call incl. NONE and vetoes, with features_snapshot JSON, explanation, mode dry_run/demo/live), orders (intents, results, retcodes, slippage = intended vs filled price), managed_actions (trade-manager rule firings), trades (full lifecycle: signal_id -> entry deal -> management -> exit deal, net P&L after all costs, MAE/MFE from bar data), equity_snapshots, system_events (state transitions, halts, errors, reconciliations). Everything queryable by time range + instrument; document the schema in CLAUDE.md.
 2. analytics.py -> reports/forward_test_{period}.html:
 - Execution quality: realized spread + slippage distributions vs the backtester's cost model assumptions (THE key go-live input - flag if live costs exceed modeled costs).
 - Performance: equity curve vs the walk-forward Monte Carlo cone (Prompt 9) - are we inside the expected band? Win rate / expectancy / PF with confidence intervals given N trades so far.
 - Model behavior: live P_long/P_short distributions vs training-time distributions (drift); veto/rejection frequency by reason; decision latency stats.
 - Discipline audit: any order not preceded by an approved decision (must be zero), any risk rejection overridden (must be zero).
 3. scripts/journal_report.py CLI to generate the report for a date range; add a go-live checklist section that auto-fills the measurable items from Chapter 10 (weeks run, trade count, PF, DD, cost deviation) with PASS/FAIL against config thresholds.
 4. Retention: journal is append-only; add an export-to-Parquet snapshot command for offline research.
 5. tests: trade lifecycle stitching from order/deal fixtures (incl. partial closes); MAE/MFE computation; cost-comparison math; checklist evaluation logic.
- Definition of done: pytest passes; after a multi-day dry run, journal_report.py produces a complete report; the go-live checklist renders with honest FAILs (we have not run 12 weeks yet - it should say so).

PROMPT 17 OF 20 · PHASE F — OPERATIONS

Prompt 17 — Monitoring: Telegram bot and dashboard

Eyes and hands on the system from your phone: push alerts, remote status/halt/resume, and a local Streamlit dashboard.

Paste this into Claude Code:

Read CLAUDE.md first. Build danalite/monitor/telegram_bot.py, notifier.py, dashboard.py.

1. notifier.py: a simple Notifier interface used by orchestrator/risk/gateway (they already have stub hooks): notify(level, title, body). Telegram backend via python-telegram-bot with a bot token + chat id from env vars (document the 2-minute BotFather setup in README). Levels: INFO (trade opened/closed with P&L and explanation), WARNING (halts, data staleness, reconciliation findings), CRITICAL (crash, drawdown breaker, repeated order failures). Rate-limit and batch INFO messages; CRITICAL always immediate. Graceful no-op if unconfigured.
 2. telegram_bot.py: command handler (whitelisted chat id ONLY, reject others): /status (state, equity, open positions, today's P&L, data freshness, heartbeat age), /positions (detail incl. current SL/TP), /halt and /halt_flat (create kill-switch files via the Prompt 15 mechanism), /resume (explicit resume), /report (last daily digest again). Runs inside the orchestrator process or as a sidecar - pick simpler and justify in a comment.
 3. Daily digest (scheduled 21:00 UTC): equity + day P&L, trades with one-line explanations, current open risk, risk-limit headroom (how close to daily/weekly limits), decisions count by outcome, collector/news health, disk space. Sent via Telegram and saved to reports/digests/.
 4. dashboard.py: Streamlit app reading ONLY from SQLite/Parquet (never touches the gateway): equity curve vs Monte Carlo cone, open positions, last 50 decisions with explanations and veto reasons, forward-test-vs-backtest cost panel (from Prompt 16 analytics), feature drift sparklines, collector heartbeat status. Auto-refresh; runs on localhost via scripts/run_dashboard.py.
 5. tests: notifier formatting + rate limiting with a fake transport; command auth rejection; digest content assembly from journal fixtures.
- Definition of done: pytest passes; I receive a real Telegram message from a test command; /status answers correctly during a dry run; dashboard renders against my current journal data.

PROMPT 18 OF 20 · PHASE F — OPERATIONS

Prompt 18 — Continuous learning: retraining pipeline

Scheduled retraining with champion/challenger promotion gates and automatic rollback — how the system improves from its own accumulating history without destroying itself.

Paste this into Claude Code:

Read CLAUDE.md first. Build danalite/models/retrain.py + scripts/run_retrain.py.

1. retrain.py pipeline (monthly by default, also triggerable manually or by drift): (a) rebuild the dataset (Prompt 6) extended through yesterday, including the news features that now have REAL collected history; (b) retrain per Prompt 7 with the frozen champion hyperparameters (no Optuna in scheduled runs - full re-tuning stays a manual, deliberate act); (c) evaluate the challenger on a HOLDOUT of the most recent 8 weeks that neither champion nor challenger trained on, PLUS the challenger must replay the walk-forward test protocol; (d) promotion gate: challenger promotes only if holdout expectancy $\geq$ champion's AND max drawdown $\leq$ champion's * 1.1 AND calibration error not worse. Otherwise keep champion; either way write reports/retrain_{date}.md with the comparison and decision rationale.
 2. Registry integration: promotions update the champion pointer atomically; the orchestrator loads models by champion pointer at startup and on a /reload command (safe hot-swap only in HALTED/no-open-positions state - document why).
 3. Post-promotion probation: for 2 weeks after any promotion, the journal analytics (Prompt 16) compare live expectancy vs the challenger's holdout expectancy; if it degrades beyond a configured band, auto-rollback to the previous champion and notify CRITICAL via Telegram.
 4. Drift trigger: weekly job computes PSI (population stability index) between training feature distributions and the last 4 weeks of live feature snapshots from the journal; PSI above threshold on core features -> WARNING notification recommending early retrain (never auto-retrain on drift alone - a human stays in this loop).
 5. Windows Task Scheduler setup script/instructions for the monthly run.
 6. tests: promotion gate logic on synthetic metric fixtures (all branches); rollback trigger; PSI computation vs hand-computed values; registry pointer atomicity.
- Definition of done: pytest passes; a full retrain run executes end-to-end on existing data and produces the report with a defensible promote/keep decision.

PROMPT 19 OF 20 · PHASE F — OPERATIONS

Prompt 19 — Capital policy: scaling, set-aside and withdrawals

The module that turns trading results into disciplined financial decisions: equity-tier scaling, profit set-aside, and monthly withdrawal recommendations.

Paste this into Claude Code:

Read CLAUDE.md first. Build danalit/risk/capital.py + its monthly report.

1. Equity tiers (config; defaults from the roadmap Chapter 9): per tier - enabled instruments, risk_per_trade, and profit set-aside percentage:
<\$50: EURUSD only, 0.75%, 0% set-aside | \$50-200: +XAUUSD, 0.75%, 10% | \$200-1000: all three, 1.0%, 25% | >\$1000: all three + shadow candidates, 1.0%, 40%.
Tier changes require N=5 consecutive trading days beyond the boundary (hysteresis - no tier flapping), apply only at day rollover, never mid-trade, and always notify via Telegram.
 2. Set-aside ledger (SQLite): at each calendar month end compute net realized P&L; if positive, credit set_aside_balance by the tier percentage. This is a VIRTUAL ledger - money stays at the broker until withdrawn - but reserved funds are excluded from the equity figure used for position sizing (risk is computed on working equity = broker equity - set_aside_balance).
 3. Withdrawal recommendation: when set_aside_balance >= a configured minimum (default \$25, broker-dependent), the monthly report recommends a withdrawal; a CLI command ``danalit capital withdraw <amount>`` records an executed withdrawal against the ledger (the actual broker withdrawal is manual - by design, no API should be able to move your money).
 4. Drawdown interaction: set-aside credits pause while equity is below the high-water mark (recover first, skim later). Monthly report shows: equity vs HWM, tier, working equity, set-aside balance, months-to-next-tier at current expectancy, and a compounding projection table (conservative/expected/optimistic scenarios from walk-forward Monte Carlo percentiles - clearly labeled as projections, not promises).
 5. Wire tier state into the risk manager (Prompt 10 already reads tier config - connect live updates) and the digest (Prompt 17).
 6. tests: tier hysteresis sequences, set-aside math across profitable/losing/mixed months, working equity exclusion, HWM pause rule, withdrawal ledger consistency.
- Definition of done: pytest passes; a simulated 12-month equity path produces a correct ledger and a readable monthly report; risk manager provably sizes from working equity.

PROMPT 20 OF 20 · PHASE F — OPERATIONS

Prompt 20 — Hardening, deployment and the go-live runbook

Failure-mode testing, Windows service deployment with a watchdog, and the operational runbook — the final prompt before the formal forward test begins.

Paste this into Claude Code:

Read CLAUDE.md first. This final build prompt hardens Danalit for unattended operation.

1. Integration test suite (tests/integration/, pytest -m integration): full pipeline against recorded fixtures - a scripted 'market day' replayed through orchestrator+engine+risk+mocked gateway asserting journal end-state; restart-during-open-position reconciliation; kill-switch honored mid-loop.
2. Failure injection tests: gateway raises timeouts/requotes/disconnects mid-sequence; SQLite locked/disk-full on journal write (orders must fail CLOSED - no journal, no order); corrupted config (refuse to start); system clock skew detection vs broker time (warn/halt thresholds); collector dead >30min during TRADING (stale-data vetoes must fire).
3. Deployment: scripts/deploy/ with (a) install_tasks.ps1 creating Windows Task Scheduler entries - MT5 terminal at logon, collector daemon at logon, trading orchestrator at logon (dry_run per config), dashboard optional, watchdog every 5 min, monthly retrain, daily digest; (b) watchdog.py: checks orchestrator + collector heartbeat files, restarts the corresponding task on staleness (with restart-storm protection: max 3 restarts/hour then CRITICAL notify and stop); (c) power settings + Windows Update deferral checklist in docs.
4. RUNBOOK.md covering, step by step: cold start; clean shutdown; resume after halt; drawdown breaker manual reset (with mandatory review questions before reset); broker maintenance windows; what every CRITICAL alert means and the first three diagnostic steps for each; demo -> live switchover procedure (env var change + checklist + deliberately starting at tier-1 risk); and 'break glass': how to flatten everything manually in the MT5 terminal if Danalit is dead.
5. A final pre-flight command: `danalit preflight` - runs config validation, gateway smoke checks (demo), DB integrity (PRAGMA integrity_check), disk space, clock sync, model registry sanity, pending-migration check; prints a PASS/FAIL table. The orchestrator refuses TRADING state if preflight fails.
6. Update CLAUDE.md + README to final state: full architecture, module map, common tasks. Definition of done: pytest incl. integration marker passes; preflight passes on my machine; install_tasks.ps1 creates tasks (I will verify by rebooting); RUNBOOK.md is complete enough that someone else could operate the system from it. The formal 12-week forward test starts after this.

14. Timeline and Milestones

Assuming 8–12 focused hours per week alongside other commitments. Claude Code compresses the coding dramatically, but data downloads, training runs, and — above all — the mandatory 12-week forward test set the floor. There is no honest way to shortcut the forward test.

Phase	Weeks	Prompts	Milestone
A. Foundation	1–2	1–3	Repo scaffolded; 10 years of price data stored; news collector running 24/7 from now on.
B. Features & labels	3–4	4–6	Leakage-safe feature matrix and labelled datasets for all three instruments.
C. Models & backtest	5–8	7–9	First honest walk-forward backtest report; go/no-go decision on the strategy's edge.
D. Decision & risk	9–11	10–13	Full decision pipeline passing simulated trading tests; optional RL exit agent evaluated.
E. Execution	12–14	14–16	System trading a demo account end-to-end, fully journaled.
F. Operations	15–16	17–20	Monitoring, retraining, capital policy and hardening complete; forward test formally begins.
Forward test	17–28	—	≥ 12 weeks, ≥ 100 trades on demo; go-live checklist evaluated.
Live (cent account)	29+	—	\$20 on a cent account at tier-1 risk settings; monthly capital policy reviews.

IF THE BACKTEST SHOWS NO EDGE AT WEEK 8

This is the most likely failure point, and it is a scheduled decision, not a surprise. Options, in order: (1) enrich features (higher-timeframe context, cross-asset features) and re-run; (2) change the trade definition (horizon, barrier multiples) — many edges exist at H1 that don't at M15; (3) narrow scope to session-specific trading (e.g. London open only); (4) accept a simpler rule-based strategy with ML as filter only (meta-labeling over a classic trend/breakout rule is a proven pattern and often beats pure-ML). The infrastructure is edge-agnostic — nothing built is wasted while you iterate on the strategy.

15. Final Words: Success Criteria and Risk Disclosure

Define success now, before emotions do it for you later. Year-one success for this project is: **(1)** a complete, tested, monitored autonomous system; **(2)** a 12-week forward test whose live behaviour matches its backtest within tolerance; **(3)** a live cent account that survives — capital preserved within the drawdown limits — while compounding modestly; **(4)** a growing proprietary news/decision dataset and a retraining loop that measurably improves the model. Hitting all four puts you in the top percentile of people who attempt this. Profit beyond that is the market's decision as much as yours; discipline is entirely yours, and this architecture enforces it.

RISK DISCLOSURE

Trading foreign exchange, metals and index CFDs on margin carries a high level of risk and is not suitable for all investors. Leveraged trading can result in losses exceeding deposits. Past performance — including backtested and demo performance — is not indicative of future results. Backtested results have inherent limitations and are prepared with the benefit of hindsight. Nothing in this document is financial advice. Never trade money you cannot afford to lose entirely. Verify that automated trading and your chosen broker are legal and regulated in your jurisdiction.